Indicații

- Testul conține 2 subiecte și durează 60 de minute. Fiecare subiect se punctează cu maxim 10 puncte pentru o rezolvare corectă și completă.
- Se pot obține punctaje parțiale pentru un subiect numai dacă acest lucru este specificat.
- Pentru a fi punctată, o rezolvare **trebuie** să includă și metoda de verificare a funcționalității acesteia.
- Pentru subiectul 2, este recomandată utilizarea macroului `PRINTF64` pentru afișarea la ecran, **doar dacă nu este explicit impusă utilizarea funcției `printf()`** de către enunț.
- Ordinea de rezolvare a subiectelor este la alegerea voastră.
- Subiectele se rezolvă pe mașina virtuală de PCLP2.

Subiecte

Subiectul 1

Maeștrii programării low-level s-au adunat astăzi să afle de la tine cum se fac cele mai interesante operații cu memoria. Dovedește-le că, într-o bună zi, vei putea face și tu parte din acest cult al programatorilor low-level.

a) [1p] Identificați care dintre simbolurile din fișierul obiect `symbols.o` NU sunt exportate și sunt inițializate cu o valoare diferită de zero.

b) [2p] Cineva tocmai a aflat cum funcționează un pointer și vrea să-ți facă o farsă. A creat un pointer care pointează către o zonă de memorie, care la rândul ei pointează către altă zonă de memorie și tot așa — practic, un lanț de pointeri! Din fericire, ultimul pointer pointează către o zonă de memorie unde se află un string.

   În fișierul `look_behind.c`, funcția `get_ptr` returnează o adresă în spatele căreia, la un moment dat, se află un string. Treaba ta este simplă: găsește adresa din memorie care pointează către acel string și apoi afișează-l!

   Hint: După ce printați stringul corect ar trebui să primiți SEGFAULT.

c) [2p] S-au strâns toți prietenii lui Linus, dar nimeni nu știe să apeleze o simplă funcție!

   În fișierul `no_caller_id.c`, funcția `get_ptr` returnează adresa unei funcții care are următorul antet:

```
char *func(uint64_t, char *)
```

   Task-ul tău este să apelezi această funcție cu argumentele `0xdeadcafebabeb00b` și `"just a simple cast"` și să afișezi stringul de adresa returnată.

d) [5p] Ultimul pas pentru a te dovedi vrednic și a intra în cultul secret este să găsești flag-ul ascuns în executabilul `obfuscate`.

   Hint: Folosiți GDB!

Subiectul 2

Batman, Superman, Flash, Arrow și Robin s-au adunat în peștera lui Batman pentru a combate infracțiunile. Tu ești aici pentru a-i ajuta să reușească în misiunea lor.

a) [2p] În timp ce supereroii încearcă să plece din Batcave pentru a lupta împotriva infracțiunilor, automobilul lui Batman nu pornește. Rezolvă *segmentation fault-ul* din fișierul `segway.asm`. ATENȚIE! Nu modifica funcția main!

b) [4p] Supereroii au ajuns la locul infracțiunii, unde Doctor Strange i-a legat pe toți cu un lanț. Batman este legat de un stâlp, iar de el este legat un alt erou și tot așa. În fișierul `linked.asm` avem structura `hero`, unde *codename* reprezintă porecla acelui supererou, iar *next* este adresa eroului de care este legat (o listă simplu înlănțuită). Începând de la Batman, parcurgeți lista până ajungeți la ultimul erou (câmpul *next* va indica NULL), afișând pentru fiecare valoarea câmpului *codename*.

c) [4p] Acum că am aflat porecla fiecărui supererou, trebuie să îi ajutăm să scape din lanțuri. În fișierul `unchained.asm` avem șirul de caractere `key`, care a fost criptat. Pentru a debloca lacătul, va trebui să faceți XOR pe fiecare byte cu `0x67`, iar apoi să interschimbați primii 4 biți cu ultimii 4 biți. Afișați apoi mesajul obținut.

Exemplu de interschimbare: 1010 1111 → 1111 1010